PARKING LTD.

CAPSTONE MANUAL

TECHNICAL MANUAL

Architecture, data model, API boundaries, computer vision, business rules, and operational limits.

Automated Parking Management System

Group code: CAP-2520-IT

Prepared 9 November 2026 | Version 1.0

For the evaluation panel, developers, technical maintainers, and system administrators.

1. System summary and implementation boundary

Parking Ltd. is a local, web-based parking management demonstration. The frontend is static HTML/CSS/JavaScript, the API is PHP, persistent state is SQLite, and gate/sectional vision processing is Python with OpenCV, Ultralytics YOLO, and EasyOCR. The application database is backend/database/parking.db. Recorded and live-gate demonstrations both publish into the same API and can change the same main database.

Layer	Current implementation
Web client	Role-gated pages under frontend/; browser calls endpoints.php over same-origin HTTP.
Application/API	PHP 8+ router and backend/api/endpoints.php; server-side sessions, role checks, CSRF for authenticated writes.
Persistence	SQLite relational DB, schema in backend/database/Schema.sql; operational schema upgrades in API integration code.
Gate vision	computer_vision/live_vigi.py; shared PlateDetector and GateColorObserver; NVR channels 1/13 or recorded gate clips.
Sectional vision	computer_vision/sectional_occupancy.py; bay model, generic vehicle detector, calibrated polygons, repeated plate/color samples.
Simulation orchestration	computer_vision/full_simulation.py and recorded_test_cases/*.py; recorded gates or live gates plus recorded occupancy.

> **Hardware integration boundary**
>
> The current repository records digital entry/exit clearance. It does not implement a tested relay, barrier actuator, radar loop, or safety interlock. A person operates the physical barrier. Do not describe the software as automatically opening the physical gate.

2. Deployment architecture and event path

Both CV and browser access the PHP API. The CV process sends camera events with a shared X-Exit-Event-Token header. Authenticated staff actions use PHP sessions and CSRF protection. Application timestamps are stored in UTC; pages present local Philippine time.

Step	Input -> processing -> persistent state
1. Entrance	Channel 1 or recorded clip -> plate/color observation -> camera_entry_event -> vehicle/session, entry token, occupied assigned slot, audit log.
2. Sectional	Snapshot/parking video -> bay localization and vehicle occupancy -> sectional_event -> slot_observations, occupancy counters, evidence, optional alerts.
3. Business action	Driver reservation, cashier bill, tenant validation, or Admin alert decision -> business endpoint -> payment/reservation/violation/audit state.
4. Exit	Channel 13/recorded plate or issued QR fallback -> payment, identity, color, alert review checks -> close session, free assigned slot, audit.
5. Frontend	Admin, Cashier, Driver, or Vendor reads the same SQLite-backed API; refresh to see new state.

3. Data model

Table	Role in current app
accounts	Login identity, role, registered plate, is_vip priority flag.
vehicles	Normalized vehicle plates and vehicle type. QR-only entries have a generated internal QR_ONLY identifier.
parking_slots	Physical bay number, zone, active state, and current occupied flag.
spatial_allocation	Zone capacity and active parked/reservation counts used by map/allocation.
parking_sessions	Entry/exit times, assigned slot, payment state, entry QR token, color/confidence, exit method.
reservations	Vehicle, reserved bay, status, paid/admin-exempt state, validity dates, QR token.
reservation_payments	Local simulator payment state/amount; no payment provider is contacted by the local checkout.
violations	Session-linked penalty amount/status. Ordinary confirmed penalty is PHP 200; lost-ticket fee is PHP 100.
audit_logs	Operational actions and authenticated actor details.
simulation_sessions, slot_observations, sectional_events, sectional_alerts	Simulation binding, camera occupancy evidence, event outcomes, staff review decision and evidence path.
gate_color_observations	Repeated entrance/exit camera color observations supporting a session.

4. State transitions and authorization

Object	Typical state path	Guard
Parking session	PARKED / UNPAID -> payment status cleared -> EXITED	Plate or issued QR lookup; payment, outstanding fines, identity/color review, and pending alert rules.
Reservation	PENDING_PAYMENT -> ACTIVE/PAID -> FULFILLED	Slot hold expires after 10 min if unpaid. Entry must match assigned reservation unless authorized admin action.
Sectional alert	PENDING_REVIEW -> CONFIRMED or REJECTED	Admin review. Confirmation creates/link a fine; rejection has no fee.
Slot	VACANT -> OCCUPIED -> VACANT	Entry and session owns assigned bay; sectional camera observation may mirror documented simulation occupancy.

5. Computer vision pipelines

Gate ALPR and color

live_vigi.py opens VIGI channel 1 (entrance) and channel 13 (exit), or receives file sources in the recorded runner. PlateDetector uses YOLO vehicle/plate boxes, track/motion context, image crops and EasyOCR. Candidate plate strings are normalized and require repeated agreement before publishing. Entrance search tolerates wider/less frontal angles than exit. GateColorObserver samples a simple color class across frames; at least three agreeing readings determine the stored label or UNKNOWN. The observed color is supporting identity evidence, never an exit credential.

Sectional occupancy and identity

The uploaded sectional checkpoint locates slot/bay regions. A generic vehicle model detects cars; calibrated zone JSON defines bay polygons and roadway/restricted polygons. Occupancy is vehicle-in-polygon observation, not training-set occupancy. Pre-entry snapshots evaluate the selected zone; unavailable or unknown bay state is not equivalent to vacant. Parking clips stop after the configured continuous hold (default 10 video seconds). Sectional plate OCR and color may be recorded when the view supports repeated reads.

The code labels clip identity as a simulation assertion; it is not an independent OCR result. When a clip is unrelated to the active session, the system must leave plate-to-bay identity unknown and must not raise a plate-specific mismatch solely from a clip label.

QR fallback

Every successful entry receives an opaque random QR token, including entries with a readable plate. For unreadable plate walk-in, the live detector can issue a QR-only session; active paid reservation entry can instead accept the reservation QR to recover its registered plate/bay. At exit, the issued entry QR is the fallback credential. Color only assists staff review.

6. Business rules in the implemented build

Rule	Current behavior
Allocation	Eligible vacant slots are chosen from active database state, preference, priority and reservations. A is VIP/PWD priority; standard walk-ins prefer B then C/D/E/F. F is allocation-count based, not sectional-camera observed.
Zone A simulation	Snapshot-visible existing vehicles are assumed authorized for the simulation. A new non-VIP entrant is allocated outside A; if the test clip shows it parking in A, a staff eligibility alert is created after occupancy confirmation.
Grace	Up to and including 15 elapsed minutes is a free/bypassed stay. Grace begins at entry; there is no presentation countdown.
Standard daytime fee	PHP 30 flat after grace.
Overnight	PHP 100 total when the stay crosses local date or exit time is 23:00 or later.
Tenant	Authorized Joey's Vendor validation waives the PHP 30 base and allows exit within 20 minutes of validation. Outside the window the base is due; other charges remain.
Reservation	PHP 60 per day plus PHP 100 when duration is at least two days. Local demo checkout holds the slot 10 minutes and moves no money.
Lost ticket	PHP 100 extra charge.
Violation	Confirmed ordinary penalty PHP 200; rejected alerts create no penalty. Cashier settles vehicle-linked unpaid citations before exit.
Roadway enforcement	Stationary occupancy in configured driveway/double-parking zones creates evidence-backed staff review. Current sectional monitor default is 20 seconds; the Zone C recorded business-rule clip demonstrates 20 seconds.

7. API surface and access

API family	Purpose and roles
session, login, logout, register	Authentication and current role identity; login/register are public.

API family	Purpose and roles
map, reserve, active_reservation, local_checkout	Driver/Admin availability, reservation creation, local simulated checkout.
cashier_lookup, cashier_payment, exit_barrier_clearance	Cashier/Admin invoice, cash payment, digital exit decision.
vendor_lookup, vendor_validate, vendor_* reports	Vendor/Admin customer session validation and activity.
admin_* , set_vip, toggle_slot, review_sectional_alert	Administrator operational actions and reporting.
camera_entry_event, exit_camera_event, sectional_event, simulation_*	Token-authenticated computer vision and simulation events.

API authorization is enforced server-side even if a user accesses an endpoint directly. Camera event routes require POST and a shared token; the secret is read from the environment or backend/database/exit_camera_token. Authenticated POST requests use CSRF validation. Staff identity in audit entries comes from the session, not caller-supplied values.

8. Simulation and operational data

The recorded simulation runs against the same main database, and its business events are intended to remain visible to the frontend. Snapshot occupancy can synchronize slot flags. The app stores snapshots/events, audit trail, and evidence under backend/database/simulation_runs/<scenario-id>/. Do not promise automatic rollback: this build persists simulation activity. Reconcile open sessions through cashier/exit or Admin Simulation Sessions recovery.

The direct live gate mode has no sectional allocation camera command; the backend allocates using database slot state and reservation/eligibility rules. Combined live-gates mode runs a selected recorded sectional snapshot and parking clip between live entrance and exit.

9. Security, privacy, reliability

- Keep local PHP bound to loopback for development. Do not expose this demo server or its default demo accounts to a public network.
- Protect parking.db, camera token, NVR password, generated QR tokens, evidence images, and vehicle/plate records. Avoid secrets in screenshots, shell history, and source control.
- Use role-based accounts; revoke access when staff changes. Audit entries are a record of application actions, not cryptographic tamper-proof logs.
- Maintain a backup before demo sessions or schema migration. Confirm camera/API/database are all pointed at the same environment.
- Human review is required for weak plate reads, color mismatch, spatial alerts, and physical barrier operation. OCR/model outputs can be wrong under glare, occlusion, distance, motion blur, or domain shift.

10. Known limits and validation status

Area	Limit to state during review
Live NVR	RTSP connectivity depends on subnet, firmware, account, and network; software tests do not guarantee a physical site feed.
Physical gates	No verified relay/radar hardware control is present in the repository. Guard actuates the barrier.

Area	Limit to state during review
Payments	Reservation checkout is simulated; cashier operations are locally recorded. No actual payment processor is integrated.
Section F	30 spaces are counted through allocation state; no overhead sectional camera confirms them.
OCR/color	Color is coarse supporting evidence; sectional plate visibility may be insufficient. Unknown is a valid output.
Calibration	A-E region polygons are camera-specific and must be checked against actual bay markings and camera framing.
Evidence	Alert images are saved as local files beneath simulation_runs and require the host filesystem to remain available.

11. Repository reference map

Path	Purpose
frontend/	Role-based web pages, CSS, and main client API code.
backend/api/endpoints.php	API routing, authentication, cashier/vendor/admin/camera events.
backend/api/simulation_integration.php	Slot observation sync, simulation entry/exit integration, alerts, time controls.
backend/api/cashier_billing.php and exit_departure.php	Fee calculation and closed-session/slot-release transaction.
backend/database/Schema.sql	Base relational schema; parking.db contains the current local data.
computer_vision/live_vigi.py	Live NVR gates and shared detector.
computer_vision/full_simulation.py	Recorded and live-gates orchestration.
computer_vision/sectional_occupancy.py	Sectional detection, occupancy, color/plate evidence, enforcement monitor.
computer_vision/recorded_test_cases/	Rule entry points, zone profiles, calibrated regions, reusable business-rule runs.
docs/	Current installation, live VIGI, simulation, local payment, and event-schema notes.